

GrocerIQ

Grocery Price Tracker & SmartShoppingList

CS Term Project | Preliminary Report (prelim.pdf)

1. Project Title

GrocerIQ — A Grocery Price Tracker and Smart Shopping List Web Application

2. Problem Description and Domain

Grocery prices fluctuate frequently across different stores and over time, yet most shoppers have no easy way to compare historical prices, spot genuine sales, or make data-driven decisions about where to shop. Inflation and supply chain disruptions have made this problem more acute, with consumers increasingly looking for ways to stretch their grocery budgets.

The domain is consumer grocery retail. Our application targets everyday shoppers who want to track prices at stores near them, manage their shopping lists, and receive automated alerts when prices drop below their historical average. The application also incorporates a simple linear regression model to predict near-term price trends for individual products.

3. Solution Description and User Interfaces

GrocerIQ is a Spring Boot web application backed by a MySQL database. It provides five distinct user interfaces, each driven by live database queries:

UI 1 — Signup / Login Users register with a username, email, and password. Passwords are hashed with BCrypt before storage — the plaintext password is never saved. On login, the entered password is compared against the stored hash using Spring Security. A session is created upon successful authentication and destroyed on logout.

UI 2 — Home Dashboard The landing page after login. Displays a curated set of featured products pulled from the database. Also shows a Price Alerts section: items whose current recorded price is below their 12-month rolling average are flagged as on sale. The average is computed with a GROUP BY SQL aggregation query. A simple linear regression trend line is shown for each alerted product to indicate whether prices are rising or falling.

UI 3 — Grocery List Manager Users can create multiple named shopping lists (e.g. 'Weekly Run', 'Costco Trip'). Within each list they can add products, set quantities, and check items off as purchased. Lists can be renamed or archived. This page drives the most database writes: INSERT when adding an item, UPDATE when checking it off or changing quantity, and DELETE when removing an item.

UI 4 — Product Search and Filter Page: A full, searchable, and filterable catalog of all products in the database. Users can search by product name, filter by category (e.g., Dairy, Produce, Meat), and filter by store chain. Each result shows the current price range across tracked stores. Users can add any product directly to one of their shopping lists from this page.

UI 5 — Product Detail Page: A deep-dive view for a single product. Shows: (1) the current price at every tracked store, generated by a JOIN across the products, price_records, and stores tables. (2) A monthly price history chart going back up to 24 months. (3) The 52-week high and low prices, computed with MIN and MAX aggregation with a date range filter. (4) A predicted price for next month generated by applying simple linear regression to the product's historical price data.

UI 6 — Community Forum: A global post board where any logged-in user can share deals, tips, or questions. Users can create new posts with a title and body. All posts are visible to every logged-in user, displayed in reverse chronological order. Each post displays the author's username and the time it was posted.

4. Preliminary ER Diagram

The diagram below describes the seven entity sets and their relationships. All relationships are one-to-many (1:N). Foreign keys on the 'many' side enforce referential integrity. ListItem is a weak entity — it cannot exist without a parent ShoppingList.

Entity	Key Attributes	Relationships
User	user_id (PK, auto-increment), username (unique), email (unique), password_hash, created_at	- One User → many ShoppingLists. A User does not need to own any lists. One User → many Posts. A User does not need to post anything.
ShoppingList	list_id (PK, auto-increment), user_id (FK -> User), list_name, created_at, is_active	one shopping list → many ListItem shoppingList → one user
ListItem (weak)	item_id (PK, auto-increment), list_id (FK -> ShoppingList), product_id (FK -> Product), quantity, checked	- Every ListItem must belong to exactly one ShoppingList (list_id NOT NULL). - Every ListItem must reference exactly one Product (product_id NOT NULL). - ListItem cannot exist without a parent ShoppingList.
Product	product_id (PK, auto-increment), product_name, category, brand (nullable), unit_size, unit_type	- One Product can appear in many ListItems across different users' lists. - One Product can have many PriceRecords across stores and months.
Store	store_id (PK, auto-increment), store_name, chain, city, state, zip_code	- One Store can have many PriceRecords (one per product per month). - A Store may exist with no price records yet (e.g. newly added store).
PriceRecord	record_id (PK, auto-increment), product_id (FK -> Product), store_id (FK -> Store), price, reg_price, is_sale, price_date	- Every PriceRecord must reference exactly one Product (product_id NOT NULL). - Every PriceRecord must reference exactly one Store (store_id NOT NULL). - price_date stores the first day of the recorded month (e.g. 2024-01-01). - is_sale flags whether the price is below the regular shelf price (reg_price)
Post	post_id (PK, auto-increment), user_id (FK -> User), title, body, posted_at	- Every Post must belong to exactly one User (user_id NOT NULL). - A User does not need to have written any Posts.

Cardinality summary: User 1→N ShoppingList, ShoppingList 1→N ListItem, Product 1→N ListItem, Product 1→N PriceRecord, Store 1→N PriceRecord, User 1→N Post. On every 'many' side, the foreign key is NOT NULL — orphan records are disallowed.

5. Technologies

Layer	Technology	Purpose
Backend	Java 17 + Spring Boot 3	Application server, MVC framework, dependency injection
Database	MySQL 8 (Docker)	Relational database; matches class project environment
DB Access	JDBC (PreparedStatement)	SQL execution; prevents injection; matches project 2 pattern
Frontend	HTML/CSS	Server-side templating for all five UI pages
Security	BCrypt (Spring Security)	Password hashing with salted BCrypt (cost factor 10)
ML/Stats	Java (simple linear reg.)	Price trend prediction; 52-week high/low; sale detection
Build	Maven	Dependency management and project build
Data	Generated/BLS DB	Generates 1000+ synthetic PriceRecord rows for demo

6. Planned Query Overview

The application will use at least 6 independent SQL statements. At least 3 will include aggregation or _____ joins:

- **AVG price per month (agg):** Dashboard price alerts — GROUP BY product_id, MONTH(price_date). Items whose current price is below a certain % of their rolling 12-month average are flagged as on sale.

Page URL: /dashboard

- **Product detail JOIN (join + agg):** Used on the Product Detail page. Joins the products, price_records, and stores tables to return each store's most recent recorded price for the selected product. Displayed as a side-by-side store price comparison.

Page URL: /product/{id}

- **52-week high / low (agg):** MIN(price) and MAX(price) with a WHERE price_date >= DATE_SUB(NOW(), INTERVAL 1 YEAR). Displayed on the Product Detail page.

Page URL: /product/{id}

- **INSERT list item:** Grocery List page — user adds a product to their list. Uses a

Page URL: /list/{id}/add

PreparedStatement INSERT into list_items.

- **DELETE list item:** Grocery List page — user removes an item. Uses a PreparedStatement DELETE by item_id + list_id (security: verify ownership).

Page URL: /list/{id}/remove

- **UPDATE checked status:** Grocery List page — user checks off a purchased item. UPDATE list_items SET checked = TRUE WHERE item_id = ?.

Page URL: /list/{id}/check

7. Indexing Plan

Two indexes will be created to optimize query performance (measured before and after population with 1000+ rows for perf.txt):

- **idx_price_product_date** on price_records(product_id, price_date) — optimizes the AVG/MIN/MAX aggregation queries and the 52-week range filter, which are the most frequently executed queries in the application.
- **idx_user_email** on users(email) — optimizes the login lookup query which filters by email on every login attempt.